

Parcial de Organización de Computadores: Diseño de la Unidad Aritmético-Lógica (ALU)

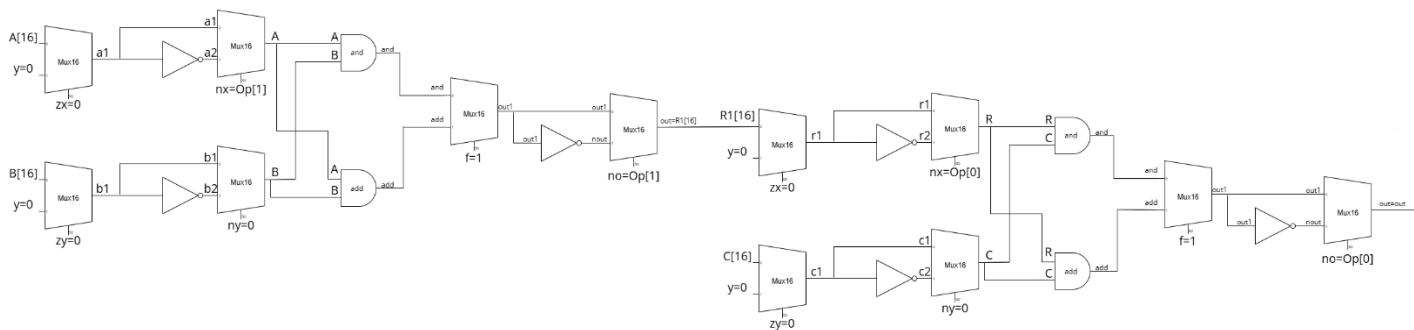
Integrantes: Cesar Montoya Giraldo, Isabela Mendoza Monsalve, Santiago Ramírez Ramírez.

Enunciado: Diseñe una Unidad Aritmético-Lógica (ALU) que cumpla con los requerimientos especificados en **UNA** de las siguientes opciones. La elección de la opción deberá ser indicada claramente en la entrega.

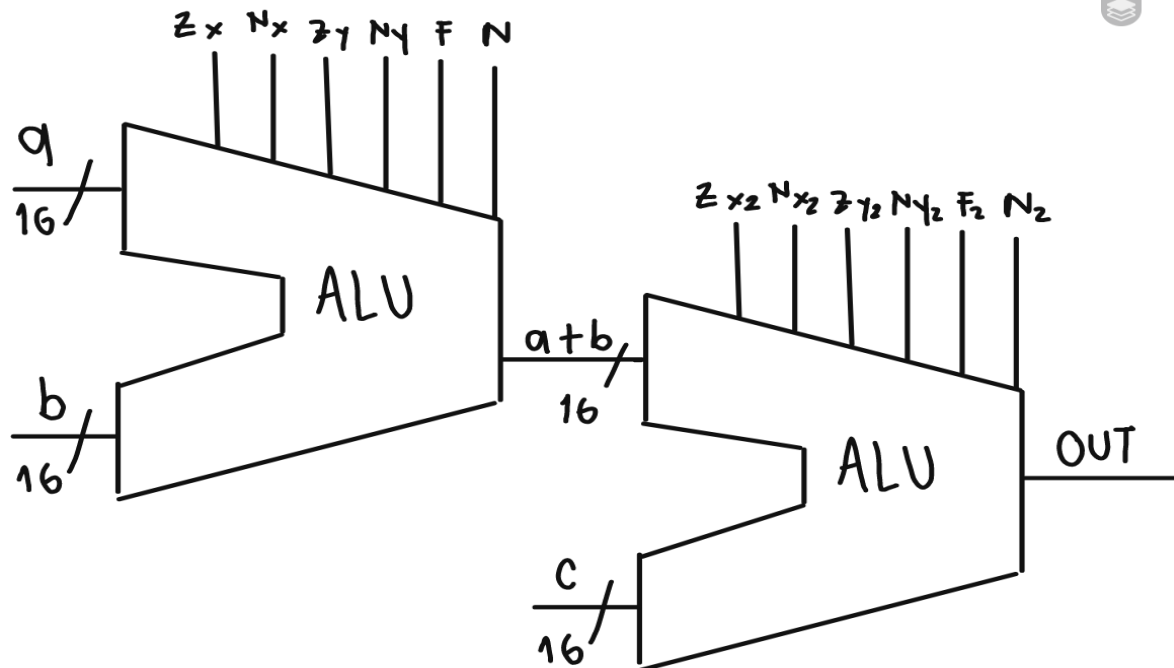
Opción 1: ALU para Operaciones Aritméticas con 3 Variables

- **Descripción:** Diseñe una ALU que realice operaciones aritméticas de suma y resta entre tres variables (A, B y C).
- **Operaciones:**
 - Op = 00: Resultado = A + B + C
 - Op = 01: Resultado = A + B - C
 - Op = 10: Resultado = A - B + C
 - Op = 11: Resultado = A - B - C
- **Entregables:** Diagrama de bloques, diseño detallado de la unidad aritmética, tabla de verdad, lógica de control y justificación de las decisiones de diseño.

Diagrama de bloques



Diseño detallado de la unidad aritmética



$$\begin{aligned} Zx &= 0 \\ Nx &= OP[1] \\ Zy &= 0 \\ Ny &= 0 \\ F &= 1 \\ No &= OP[1] \end{aligned}$$

$$\begin{aligned} Zx2 &= 0 \\ Nx2 &= OP[0] \\ Zy2 &= 0 \\ Ny2 &= 0 \\ F2 &= 1 \\ No &= OP[1] \end{aligned}$$

Tabla de verdad

Op[1]	Op[1]	Operación Realizada
0	0	$(A+B)+C$
0	1	$(A+B)-C$
1	0	$(A-B)+C$
1	1	$(A-B)-C$

Lógica control

La lógica de control está definida por el vector Op[2].

- Op[1] controla la primera ALU.
- Op[0] controla la segunda ALU.

Y la Lógica detrás de esto consiste en:

Primera ALU

- Si Op[1] = 0, la primera ALU hace una suma ($A + B$).
- Si Op[1] = 1, la primera ALU hace una resta ($A - B$).

Donde la configuración de señales es la siguiente:

Configuración de señales:

- $zx=false$
- $nx=Op[1]$: Invierte A si Op[1] es verdadero.
- $zy=false$:
- $ny=false$
- $f=true$: Selecciona la operación de suma.
- $no=Op[1]$: Invierte la salida si Op[1] es verdadero.

El resultado de las operaciones se almacena en R1.

Segunda ALU

- Si la Op[0] = 0, la segunda ALU va a realizar la operación ($R1 + C$)
- Si la Op[0] = 1, la segunda ALU va a realizar la operación ($C - R1$)

Donde la configuración de señales es la siguiente

Configuración de señales:

- $zx = false$: No se anula el operando x (R1).
- $nx = Op[0]$: Invierte R1 si Op[0] es 1.
- $zy = false$: No se anula el operando y (C).
- $ny = false$: No se invierte C.
- $f = true$: Selecciona la operación de suma.
- $no = Op[0]$: Invierte la salida si Op[0] es 1.

Justificaciones:

La implementación de 2 ALU surgió de la idea de que una sola ALU puede sumar y restar 2 variables A Y B por lo que, como resultado obtenemos una variable auxiliar ($R1=A\pm B$), y

necesitamos sumar o restar una tercer variable C, por lo cual se optó por implementar una segunda ALU, la cual recibiría como entradas R1 y C y así obtener las operaciones de $(R1 \pm C)$, consiguiendo así las operaciones aritméticas entre las variables A, B y C, es decir, $(A \pm B \pm C)$.

Con esta idea y observando el funcionamiento de la ALU y su programación en Nan2Tetris que fue realizada en clase, se hizo uso de las compuertas lógicas y su estructura, para ser implementadas en el proyecto y adaptadas a los requisitos propuestos, dando así solución al reto.

Referencias:

Video de apoyo: <https://youtu.be/mi9ZeXVCZOM?si=fKGtCnbapqVA5X-v>

Diagrama hecho con: [Untitled | Visual Paradigm Online](#)

Guía: [NAND2Tetris](#)